

Boost C++ Kütüphaneleri

Kapsamlı Uygulamalı Rehber

Modern C++ için Endüstri Standardı Kütüphane Koleksiyonu

Bu belge, Boost kütüphanelerini modern C++ (C++17/20) ile, her konuyu **derlenebilir kod örnekleriyle**, mimari açıklamalarla ve pratik kullanım desenleriyle adım adım anlatır. Akıllı işaretçilerden Boost.Asio ile ağ programlamaya, metaprogramlamadan graf algoritmalarına kadar en çok kullanılan Boost bileşenlerini kapsar.

Kapsam — Kısım 1 (Temeller): Boost felsefesi • kurulum • b2 & CMake entegrasyonu • header-only vs derlenmiş.

Kapsam — Kısım 2 (Bellek & Yardımcılar): akıllı işaretçiler • Optional • Variant • Any • lexical_cast • Format • String Algo • Tokenizer.

Kapsam — Kısım 3 (Konteynerler): Container • MultiIndex • Bimap • CircularBuffer • Intrusive • DynamicBitset.

Kapsam — Kısım 4 (Eşzamanlılık & Ağ): Thread • Atomic • Lockfree • Asio (detaylı) • Fiber.

Kapsam — Kısım 5 (Metin & Veri): Regex • Spirit • PropertyTree • JSON • Serialization.

Kapsam — Kısım 6 (Sistem, Matematik & Graf): Filesystem • DateTime • ProgramOptions • Interprocess • Math • Multiprecision • Graph (BGL).

Hazırlanma Tarihi: 19 Temmuz 2026

Boost 1.85+ • C++17 / C++20 • g++ / clang++

Contents

1 Boost Nedir ve Neden Kullanılır?	2
1.1 Boost'u Ne Zaman Kullanmalı, Ne Zaman Kullanmamalı?	2
1.2 Header-Only ve Derlenmiş Kütüphaneler	3
2 Kurulum ve Yapılandırma	3
2.1 Paket Yöneticisiyle Kurulum (En Kolay Yol)	3
2.2 Kaynaktan Derleme (b2 / Boost.Build)	4
2.3 Elle Derleme: g++ Komut Satırı	4
2.4 CMake ile Boost Kullanımı (Önerilen Yöntem)	5
2.5 İlk Boost Programı	5
3 Akıllı İşaretçiler (Boost.SmartPtr)	7
3.1 Standartta Bulunmayan Boost Akıllı İşaretçileri	7
3.2 intrusive_ptr: Gömülü Referans Sayacı	7
3.3 Boost.Pool: Havuz Tabanlı Bellek Ayırma	8
4 Boost.Optional: Değeri Olmayabilen Nesneler	9
5 Boost.Variant ve Variant2: Tip Güvenli Birleşimler	10
6 Boost.Any: Herhangi Bir Tipi Tutmak	11
7 Metin İşleme Yardımcıları	12
7.1 Boost.Lexical_cast: Basit Tip Dönüşümleri	12
7.2 Boost.Format: Tip Güvenli printf	12
7.3 Boost.String_algo: Metin Algoritmaları	13
7.4 Boost.Tokenizer: Esnek Belirteçleme	14
8 Boost.Container: Gelişmiş Konteynerler	15
9 Boost.MultiIndex: Çok İndeksli Konteynerler	16
10 Boost.Bimap: Çift Yönlü Haritalar	17
11 Boost.CircularBuffer: Dairesel Tampon	18
12 Boost.DynamicBitset: Dinamik Bit Kümeleri	18
13 Boost.Thread: İş Parçacığı Yönetimi	20
13.1 Karşılıklı Dışlama ve Kilitler	21
14 Boost.Asio: Asenkron Giriş/Çıkış ve Ağ	21
14.1 Temel Kavramlar: io_context ve Zamanlayıcılar	21
14.2 Asenkron TCP Yankı (Echo) Sunucusu	22
14.3 Asenkron TCP İstemcisi	24
14.4 Modern Yaklaşım: Coroutine'lerle Asio (C++20)	24
15 Boost.Atomic ve Boost.Lockfree	25
16 Boost.Regex: Düzenli İfadeler	27
17 Boost.PropertyTree: JSON, XML, INI Ayırıştırma	27

18 Boost.JSON: Modern JSON İşleme	29
19 Boost.Spirit: Ayrıştırıcı Üretici	30
20 Boost.Filesystem: Dosya Sistemi İşlemleri	31
21 Boost.DateTime ve Boost.Chrono	32
22 Boost.ProgramOptions: Komut Satırı Ayrıştırma	32
23 Boost.Interprocess: Süreçler Arası İletişim	34
24 Boost.Multiprecision: Yüksek Hassasiyetli Aritmetik	35
25 Boost.Graph (BGL): Graf Algoritmaları	36
25.1 Graf Oluşturma ve Gezinme	36
25.2 Dijkstra ile En Kısa Yol	36
26 Kapanış: Boost Ekosistemine Bakış	38

KISIM 1

Temeller, Felsefe ve Kurulum

1. Boost Nedir ve Neden Kullanılır?

Boost, C++ dünyasının en etkili ve en saygın açık kaynak kütüphane koleksiyonudur. 1998'de, C++ standart komitesinin (ISO WG21) bazı üyeleri tarafından, standarda aday olabilecek yüksek kaliteli kütüphaneleri geliştirmek ve olgunlaştırmak amacıyla kuruldu. Bugün 160'tan fazla bağımsız kütüphaneden oluşan devasa bir ekosistemdir.

Boost'un C++ standardı üzerindeki etkisi tarihseldir: bugün standart kütüphanenin (std) ayrılmaz parçaları olan pek çok bileşen önce Boost'ta doğdu, yıllarca gerçek projelerde denendi ve olgunlaştıktan sonra standarda alındı.

Boost Kütüphanesi	Standart Karşılığı	Sürüm
<code>boost::shared_ptr</code>	<code>std::shared_ptr</code>	C++11
<code>boost::function</code>	<code>std::function</code>	C++11
<code>boost::bind</code>	<code>std::bind</code> / lambdalar	C++11
<code>boost::array</code>	<code>std::array</code>	C++11
<code>boost::thread</code>	<code>std::thread</code>	C++11
<code>boost::optional</code>	<code>std::optional</code>	C++17
<code>boost::variant</code>	<code>std::variant</code>	C++17
<code>boost::any</code>	<code>std::any</code>	C++17
<code>boost::filesystem</code>	<code>std::filesystem</code>	C++17
<code>boost::string_view</code>	<code>std::string_view</code>	C++17

Bilgi

Boost'u öğrenmek, aslında C++'ın *geleceğini* öğrenmektir. Bugün Boost'ta kullandığınız birçok kütüphane, birkaç yıl sonra standardın parçası olabilir. Ayrıca standardın henüz kapsamadığı devasa bir alan (Asio, Spirit, Graph, Multiprecision, Geometry...) yalnızca Boost'ta mevcuttur.

1.1 Boost'u Ne Zaman Kullanmalı, Ne Zaman Kullanmamalı?

Kullanın: Standart kütüphanenin karşılamadığı bir ihtiyaç varsa (ağ, ayrıştırma, çok yüksek hassasiyetli aritmetik, graf algoritmaları, tarih/saat işlemleri), yeni bir standart özelliği eski bir der-

leyicide kullanmanız gerekiyorsa (C++17 optional'ı C++14 projesinde), veya savaşta denenmiş, iyi test edilmiş bir çözüm istiyorsanız.

Dikkatli olun: Standart karşılığı zaten varsa ve C++17+ hedefliyorsanız, önce std sürümünü tercih edin. Boost bağımlılığı derleme süresini uzatır, ikili boyutu artırır ve dağıtımı zorlaştırabilir. Ayrıca bazı Boost kütüphaneleri ağır şablon (template) metaprogramlama içerir ve derleme hatalarını okumak zor olabilir.

Dikkat

Boost *tek bir kütüphane değildir*; birbirinden bağımsız onlarca kütüphanenin şemsiye adıdır. “Boost’u projeme ekledim” demek genellikle yanlıştır; “Boost.Asio ve Boost.System’i ekledim” demek doğrudur. İhtiyacınız olan alt kütüphaneleri seçerek bağlayın.

1.2 Header-Only ve Derlenmiş Kütüphaneler

Boost kütüphaneleri iki gruba ayrılır ve bu ayrım kurulum/derleme açısından kritiktir:

- **Header-only (yalnızca başlık):** Tüm kodu başlık dosyalarında bulunur. #include etmeniz yeterlidir, ayrıca bağlamanız (link) gerekmez. Çoğunluk buradadır: Optional, Variant, MPL, Hana, Spirit, Lexical_cast, Format, TypeTraits, Bind, Function, Signals2 (kısmen)...
- **Derlenmiş (compiled/separately built):** Önceden derlenmiş bir .a/.so kütüphaneye bağlanmayı gerektirir. Bunlar: System, Filesystem, Thread, Regex, ProgramOptions, DateTime (kısmen), Serialization, Log, Chrono, Context, Coroutine, IOStreams, Locale, Test (kısmen).

İpucu

Bir kütüphanenin bağlanma gerektirip gerektirmediğini bilmiyorsanız, Boost dokümantasyonundaki her kütüphanenin “Getting Started” sayfasına bakın. Genel kural: dosya sistemi, iş parçacığı, ağ, günlüklenme gibi *işletim sistemiyle etkileşen* kütüphaneler genelde derlenmiştir.

2. Kurulum ve Yapılandırma

2.1 Paket Yöneticisiyle Kurulum (En Kolay Yol)

Çoğu Linux dağıtımında Boost paketlenmiştir. Geliştirme başlıkları için -dev paketini kurmanız gerekir:

```

1 # Debian / Ubuntu
2 sudo apt update
3 sudo apt install libboost-all-dev
4
5 # Fedora
6 sudo dnf install boost-devel
7
8 # Arch Linux / CachyOS
9 sudo pacman -S boost
10
11 # macOS (Homebrew)
12 brew install boost
13
14 # Sürümü doğrulama
15 dpkg -s libboost-dev | grep Version    # Debian ailesi
16 cat /usr/include/boost/version.hpp | grep BOOST_LIB_VERSION

```

2.2 Kaynaktan Derleme (b2 / Boost.Build)

En güncel sürümü veya özel yapılandırma istiyorsanız kaynaktan derleyin. Boost, b2 (eski adıyla bjam) adında kendi derleme sistemini kullanır:

```

1 # 1) Kaynağı indir ve aç
2 wget https://archives.boost.io/release/1.85.0/source/boost_1_85_0.tar.gz
3 tar xzf boost_1_85_0.tar.gz
4 cd boost_1_85_0
5
6 # 2) b2 aracını hazırla (bootstrap)
7 ./bootstrap.sh --prefix=/usr/local --with-toolset=gcc
8
9 # 3) Derle ve kur (sadece ihtiyacınız olan kütüphaneleri seçebilirsiniz)
10 ./b2 --with-system --with-filesystem --with-thread \
11     variant=release link=shared,static \
12     threading=multi cxxstd=17 -j$(nproc)
13
14 sudo ./b2 install          # başlıkları ve kütüphaneleri /usr/local'a kur
15
16 # TÜM kütüphaneleri derlemek isterseniz --with-* bayraklarını kaldırın
17 ./b2 -j$(nproc) cxxstd=17

```

b2 seçeneği	Anlamı
variant=release	Optimizasyonlu (-O2 -DNDEBUG) sürüm
variant=debug	Hata ayıklama sembolleriyle
link=static	Statik kütüphane (.a) üret
link=shared	Paylaşımlı kütüphane (.so) üret
threading=multi	İş parçacığı güvenli sürüm
cxxstd=17	C++17 standardıyla derle
-j\$(nproc)	Tüm çekirdekleri kullan (paralel)
-with-X	Yalnızca X kütüphanesini derle
-build-dir=/tmp/b	Ara çıktı dizini

2.3 Elle Derleme: g++ Komut Satırı

Header-only bir kütüphane kullanıyorsanız, tek yapmanız gereken başlık yolunu vermektir:

```

1 # Header-only (ör. Boost.Optional) -- bağlama YOK
2 g++ -std=c++17 -I/usr/local/include main.cpp -o program
3
4 # Derlenmiş kütüphane (ör. Filesystem + System) -- bağlama VAR
5 g++ -std=c++17 main.cpp -o program \
6     -lboost_filesystem -lboost_system
7
8 # Özel kurulum yolu belirtme (kaynaktan derlediyseniz)
9 g++ -std=c++17 -I/usr/local/include main.cpp -o program \
10     -L/usr/local/lib -lboost_filesystem -lboost_system \
11     -Wl,-rpath,/usr/local/lib

```

Dikkat

Bağlama sırası önemlidir: `-lboost_filesystem` kütüphanesi `-lboost_system`'e bağımlıdır. GNU ld ile bağımlı kütüphaneyi *sonra* yazın: önce `filesystem`, sonra `system`. Yeni Boost sürümlerinde `Filesystem` artık `System`'e derleme zamanı bağımlı olmayabilir, ancak eski sürümlerde bu sıralama derleme hatalarını çözer.

2.4 CMake ile Boost Kullanımı (Önerilen Yöntem)

Gerçek projelerde Boost'u CMake üzerinden bulmak en temiz yaklaşımdır. `find_package(Boost ...)` komutu bileşenleri bulur ve modern hedef tabanlı (target-based) bağlama sağlar:

```
1 cmake_minimum_required(VERSION 3.16)
2 project(BoostOrnek LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 17)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6
7 # İhtiyacınız olan bileşenleri açıkça listeleyin.
8 # Header-only kütüphaneler COMPONENTS listesine YAZILMAZ.
9 find_package(Boost 1.75 REQUIRED COMPONENTS
10     filesystem
11     system
12     program_options
13 )
14
15 add_executable(program main.cpp)
16
17 # Modern (target tabanlı) bağlama -- tercih edilen yol:
18 target_link_libraries(program PRIVATE
19     Boost::filesystem
20     Boost::system
21     Boost::program_options
22 )
23
24 # Header-only bir kütüphane (ör. Boost.Optional) kullanıyorsanız,
25 # başlık yolları için genel Boost hedefini bağlayın:
26 target_link_libraries(program PRIVATE Boost::headers)
```

İpucu

Boost'u CMake'in `FetchContent`'i ile de otomatik indirtip derletebilirsiniz; ancak Boost çok büyük olduğu için bu, ilk derlemeyi ciddi biçimde uzatır. Büyük projelerde sistem paketini veya önceden derlenmiş bir Boost'u tercih edin. `Boost::headers` hedefi, yalnızca başlık yolu gereken header-only kütüphaneler için idealdir.

2.5 İlk Boost Programı

Boost'un doğru kurulduğunu doğrulayan minimal bir program yazalım. `Boost.Version` ile sürümü, `Boost.Lexical_cast` ile bir tür dönüşümünü test edelim:

```
1 #include <boost/version.hpp>
2 #include <boost/lexical_cast.hpp>
3 #include <iostream>
4 #include <string>
5
6 int main() {
```

```
7 // BOOST_VERSION: örn. 108500 => 1.85.0
8 std::cout << "Boost surumu: "
9         << BOOST_VERSION / 100000 << "." // ana
10        << BOOST_VERSION / 100 % 1000 << "." // ara
11        << BOOST_VERSION % 100 << "\n"; // yama
12
13 // lexical_cast: string <-> sayi guvenli donusum
14 int value = boost::lexical_cast<int>("42");
15 std::string text = boost::lexical_cast<std::string>(3.14159);
16
17 std::cout << "int: " << value << "\n";
18 std::cout << "string: " << text << "\n";
19 return 0;
20 }
```

```
1 # lexical_cast header-only olduğu için bağlama gerekmez
2 g++ -std=c++17 first_program.cpp -o first_program
3 ./first_program
4 # Çikti:
5 # Boost surumu: 1.85.0
6 # int: 42
7 # string: 3.14159
```


KISIM 2

Bellek Yönetimi ve Yardımcı Kütüphaneler

3. Akıllı İşaretçiler (Boost.SmartPtr)

Boost.SmartPtr, C++'ta otomatik bellek yönetiminin öncüsüdür. `std::shared_ptr` ve `std::weak_ptr` bu kütüphaneden doğdu. Bugün C++11+ kullanıyorsanız çoğu durumda `std` sürümlerini tercih edersiniz; ancak Boost hâlâ standartta *olmayan* önemli akıllı işaretçi türleri sunar.

3.1 Standartta Bulunmayan Boost Akıllı İşaretçileri

İşaretçi	Kullanım Amacı
<code>scoped_ptr</code>	Tek sahiplik, taşınamaz (move edilemez). Basit RAII; <code>unique_ptr</code> 'ın kısıtlı ama açık niyetli atası.
<code>intrusive_ptr</code>	Referans sayacı <i>nesnenin içinde</i> tutulur. Bellek dostu; harici kontrol bloğu yoktur.
<code>local_shared_ptr</code>	Tek iş parçacıklı <code>shared_ptr</code> ; atomik olmayan sayaç kullanır, daha hızlıdır.
<code>shared_array</code>	Dizi için <code>shared_ptr</code> (C++17 öncesi <code>shared_ptr<T[]></code> yokken kritikti).

3.2 intrusive_ptr: Gömülü Referans Sayacı

`intrusive_ptr`, referans sayacını kontrol nesnesinde değil, işaret ettiği nesnenin *kendi içinde* tutar. Bu, her nesne için ayrı bir kontrol bloğu tahsis etmeyi önler; çok sayıda küçük nesne yönetiyorsanız belleği ve önbellek yerelliğini iyileştirir. Karşılığında, nesnenizin `sayaç` ve `intrusive_ptr_add_ref / intrusive_ptr_release` fonksiyonlarını sağlaması gerekir.

```

1 #include <boost/intrusive_ptr.hpp>
2 #include <atomic>
3 #include <iostream>
4
5 // Referans sayaci taban sinifi -- nesneye "gomulu" sayac
6 class RefCounted {
7 public:
8     RefCounted() : ref_count_(0) {}
9     virtual ~RefCounted() = default;

```

```

10
11     friend void intrusive_ptr_add_ref(RefCounted* p) noexcept {
12         p->ref_count_.fetch_add(1, std::memory_order_relaxed);
13     }
14     friend void intrusive_ptr_release(RefCounted* p) noexcept {
15         // release-acquire ile güvenli silme
16         if (p->ref_count_.fetch_sub(1, std::memory_order_acq_rel) == 1) {
17             delete p;    // son referans dustu
18         }
19     }
20 private:
21     mutable std::atomic<int> ref_count_;
22 };
23
24 class Node : public RefCounted {
25 public:
26     explicit Node(int v) : value(v) {
27         std::cout << "Dugum(" << value << ") olusturuldu\n";
28     }
29     ~Node() override {
30         std::cout << "Dugum(" << value << ") yok edildi\n";
31     }
32     int value;
33 };
34
35 int main() {
36     // Sayac gomulu oldugu icin harici kontrol blogu YOK
37     boost::intrusive_ptr<Node> a(new Node(10));
38     {
39         boost::intrusive_ptr<Node> b = a;    // sayac: 2
40         std::cout << "b->deger = " << b->value << "\n";
41     }    // b yok oldu -> sayac: 1 (nesne yasar)
42
43     std::cout << "a hala gecerli: " << a->value << "\n";
44     return 0;
45 }    // a yok oldu -> sayac: 0 -> "Dugum(10) yok edildi"

```

Bilgi

`intrusive_ptr` özellikle oyun motorlarında, referans sayımlı COM benzeri API'lerde ve milyonlarca küçük nesneli grafik/sahne ağaçlarında tercih edilir. `shared_ptr`'ın nesne başına ayrı kontrol bloğu tahsisi bu senaryolarda ciddi ek yük getirir.

3.3 Boost.Pool: Havuz Tabanlı Bellek Ayırma

`Boost.Pool`, aynı boyutta çok sayıda küçük nesneyi hızlıca tahsis/serbest bırakmak için tasarlanmış bir bellek havuzudur. `new/delete`'in her çağrıda işletim sistemine gitme maliyetini ortadan kaldırır.

```

1  #include <boost/pool/object_pool.hpp>
2  #include <iostream>
3
4  struct Particle {
5      float x, y, z;
6      float speed;
7      Particle(float px, float py, float pz)
8          : x(px), y(py), z(pz), speed(0.f) {}
9  };
10

```

```

11 int main() {
12     // object_pool: nesneleri havuzdan hizlica al/ver
13     boost::object_pool<Particle> pool;
14
15     // construct() belleği havuzdan alır ve yapıcıyı çağırır
16     Particle* p1 = pool.construct(1.f, 2.f, 3.f);
17     Particle* p2 = pool.construct(4.f, 5.f, 6.f);
18
19     std::cout << "p1: " << p1->x << ", " << p1->y << "\n";
20
21     pool.destroy(p1); // yıkıcı çağırılır, bellek havuzda kalır (yeniden kullanılır)
22     // p2 için destroy çağırmasak bile havuz yok olurken temizlenir
23     return 0;
24 } // havuz yok olurken kalan tüm nesneler otomatik temizlenir

```

4. Boost.Optional: Değeri Olmayabilen Nesneler

`boost::optional<T>`, bir değerin “var veya yok” durumunu tip güvenli biçimde ifade eder. C++17’de `std::optional` olarak standartlaştı, ancak Boost sürümü daha eski derleyicilerde çalışır ve bazı ek özelliklere sahiptir (örn. referansları sarmalayabilme).

```

1  #include <boost/optional.hpp>
2  #include <boost/optional/optional_io.hpp> // operator<< desteği
3  #include <iostream>
4  #include <string>
5  #include <map>
6
7  // Bir haritada anahtar bulunmayabilir -> optional ile ifade et
8  boost::optional<std::string> find(const std::map<int, std::string>& m, int key) {
9      auto it = m.find(key);
10     if (it == m.end())
11         return boost::none; // değer yok
12     return it->second; // değer var
13 }
14
15 int main() {
16     std::map<int, std::string> dictionary = {{1, "bir"}, {2, "iki"}};
17
18     boost::optional<std::string> r1 = find(dictionary, 1);
19     boost::optional<std::string> r2 = find(dictionary, 99);
20
21     if (r1) // bool bağlamı: değer var mı?
22         std::cout << "Bulundu: " << *r1 << "\n";
23
24     // value_or: yoksa varsayılan don
25     std::cout << "99 -> " << r2.value_or("yok") << "\n";
26
27     // get_value_or ve emplace
28     boost::optional<int> number;
29     std::cout << "Başlangıç: " << number.get_value_or(-1) << "\n";
30     number.emplace(42); // yerinde oluşturun
31     std::cout << "Sonra: " << *number << "\n";
32
33     // Boost'a özgü: referans sarmalama (std::optional yapamaz)
34     int x = 100;
35     boost::optional<int&> ref = x;

```

```

36     *ref = 200;                                // x'i degistirir
37     std::cout << "x = " << x << "\n";        // 200
38     return 0;
39 }

```

Bilgi

`boost::optional<T>` (referansa optional) standartta yoktur; bu Boost'a özgü güçlü bir özelliktir. "Belki bir nesneye başvuru" ifade etmek için ham işaretçiye (`T*`) alternatif, daha okunur bir yol sunar.

Dikkat

Bir optional'ı değeri yokken `*opt` veya `opt.value()` ile açmak (dereference) tanımsız davranıştır (Boost) / istisna fırlatır (`std::value()`). Her zaman önce `if (opt)` ile kontrol edin veya `value_or` kullanın.

5. Boost.Variant ve Variant2: Tip Güvenli Birleşimler

`boost::variant`, birden fazla tipten *tam olarak birini* tutabilen, tip güvenli bir birleşim (union) sağlar. C++17'de `std::variant` oldu. Boost ayrıca daha modern, istisna güvenli `boost::variant2::variant` sunar (asla "valueless" duruma düşmez).

```

1  #include <boost/variant.hpp>
2  #include <iostream>
3  #include <string>
4  #include <vector>
5
6  // Variant birden fazla tipten birini tutar
7  using Value = boost::variant<int, double, std::string>;
8
9  // Ziyaretci (visitor): her tip için ayrı davranış
10 struct PrintVisitor : public boost::static_visitor<void> {
11     void operator()(int i) const {
12         std::cout << "int: " << i << "\n";
13     }
14     void operator()(double d) const {
15         std::cout << "double: " << d << "\n";
16     }
17     void operator()(const std::string& s) const {
18         std::cout << "string: \"" << s << "\"\n";
19     }
20 };
21
22 int main() {
23     std::vector<Value> values = { 42, 3.14, std::string("merhaba") };
24
25     // apply_visitor: aktif tipe göre doğru operator()'u çağırır
26     for (const auto& d : values)
27         boost::apply_visitor(PrintVisitor{}, d);
28
29     // which(): aktif tipin indeksi (0-tabanlı)
30     Value d = std::string("test");
31     std::cout << "Aktif indeks: " << d.which() << "\n";    // 2
32 }

```

```

33 // get<T>: dogru tipse deger, degilse istisna
34 try {
35     int i = boost::get<int>(d); // d string tutuyor -> hata
36     (void)i;
37 } catch (const boost::bad_get& e) {
38     std::cout << "Yanlis tip: " << e.what() << "\n";
39 }
40 return 0;
41 }

```

İpucu

Variant, *toplam tipler* (sum types) veya *ayrık birleşimler* (discriminated unions) için idealdir: bir JSON değeri (sayı/metin/dizi/nesne), bir AST düğümü, veya bir durum makinesinin durumları. Ziyaretçi deseni, her yeni tip eklendiğinde derleyicinin sizi eksik durumları ele almaya zorlaması sayesinde çok güvenlidir.

Modern Alternatif

Yeni kod yazıyorsanız ve istisna güvenliği kritikse, `boost::variant2` veya `std::variant` tercih edin. Klasik `boost::variant`, bir atama sırasında istisna olursa nadiren “valueless by exception” duruma düşebilir; `variant2` bunu tasarımıyla engeller.

6. Boost.Any: Herhangi Bir Tipi Tutmak

`boost::any`, *herhangi* bir tipteki tek bir değeri tip-silinmiş (type-erased) biçimde tutar. Variant’tan farkı: önceden tip listesi belirtmezsiniz. C++17’de `std::any` oldu.

```

1 #include <boost/any.hpp>
2 #include <iostream>
3 #include <string>
4 #include <vector>
5
6 int main() {
7     // Aynı kaptaki farklı tipler (heterojen)
8     std::vector<boost::any> container;
9     container.push_back(42);
10    container.push_back(std::string("metin"));
11    container.push_back(3.14);
12
13    for (const auto& a : container) {
14        // any_cast<T>: dogru tipse deger, degilse bad_any_cast
15        if (a.type() == typeid(int))
16            std::cout << "int: " << boost::any_cast<int>(a) << "\n";
17        else if (a.type() == typeid(std::string))
18            std::cout << "string: " << boost::any_cast<std::string>(a) << "\n";
19        else if (a.type() == typeid(double))
20            std::cout << "double: " << boost::any_cast<double>(a) << "\n";
21    }
22
23    // Isaretci ile guvenli cast: yanlissa nullptr (istisna atmaz)
24    boost::any a = 100;
25    if (int* p = boost::any_cast<int>(&a))
26        std::cout << "Isaretci cast: " << *p << "\n";
27    return 0;

```

28 }

Dikkat

any, tip güvenliğini çalışma zamanına erteler ve her erişimde tip kontrolü gerektirir; ayrıca küçük olmayan nesneler için yığın (heap) tahsisi yapabilir. Tipler önceden biliniyorsa `variant` neredeyse her zaman daha hızlı ve güvenlidir. `any`'yi yalnızca gerçekten *açık uçlu* tip kümeleri için kullanın (örn. eklenti sistemleri).

7. Metin İşleme Yardımcıları

7.1 Boost.Lexical_cast: Basit Tip Dönüşümleri

`lexical_cast`, metin ile sayısal tipler arasında «/» akış operatörlerini kullanarak dönüşüm yapar. `std::to_string` veya `std::stoi`'den daha genel; herhangi bir akış-uyumlu tip için çalışır.

```

1 #include <boost/lexical_cast.hpp>
2 #include <iostream>
3 #include <string>
4
5 int main() {
6     // string -> sayi
7     int i = boost::lexical_cast<int>("123");
8     double d = boost::lexical_cast<double>("3.14");
9
10    // sayi -> string
11    std::string s = boost::lexical_cast<std::string>(2026);
12
13    std::cout << i << " " << d << " " << s << "\n";
14
15    // Hatalı girdi -> bad_lexical_cast istisnası
16    try {
17        int error = boost::lexical_cast<int>("abc");
18        (void)error;
19    } catch (const boost::bad_lexical_cast& e) {
20        std::cout << "Donusum hatasi: " << e.what() << "\n";
21    }
22    return 0;
23 }
```

7.2 Boost.Format: Tip Güvenli printf

`boost::format`, `printf`'in tip güvenli ve genişletilebilir alternatifidir. `%` operatörüyle argümanları besler; yanlış tip sorunlarını ve bellek taşmalarını önler.

```

1 #include <boost/format.hpp>
2 #include <iostream>
3 #include <string>
4
5 int main() {
6     // %1%, %2% ... konumsal argümanlar
7     std::string s = (boost::format("%1% + %2% = %3%") % 2 % 3 % 5).str();
8     std::cout << s << "\n";           // 2 + 3 = 5
9 }
```

```

10 // printf tarzı biçimlendirme belirteçleri
11 std::cout << boost::format("Pi ~ %.4f\n") % 3.14159265;
12 std::cout << boost::format("Hex: %1$#x, Onluk: %1$d\n") % 255;
13
14 // Alan genişliği ve hizalama
15 std::cout << boost::format("|%1$10s|%2$-10s|\n") % "sag" % "sol";
16
17 // Aynı argümanı tekrar kullanma (%1% birden fazla kez)
18 std::cout << boost::format("%1%^2 = %2%\n") % 7 % (7*7);
19 return 0;
20 }

```

İpucu

`boost::format`, konumsal argümanları (%1%) desteklediği için *yerelleştirmede* (i18n) çok kullanışlıdır: farklı dillerde kelime sırası değiştiğinde biçim dizesini değiştirmeniz yeterlidir, kod aynı kalır.

7.3 Boost.String_algo: Metin Algoritmaları

`Boost.String_algo`, standart kütüphanede eksik olan onlarca metin işleme fonksiyonu sunar: büyük/küçük harf, kırpma (trim), bölme (split), birleştirme (join), değiştirme (replace), arama.

```

1 #include <boost/algorithm/string.hpp>
2 #include <iostream>
3 #include <string>
4 #include <vector>
5
6 int main() {
7     namespace ba = boost::algorithm;
8
9     std::string s = " Merhaba, Dünya ";
10
11     // Kırpma (boslukları kaldır) -- yerinde (in-place)
12     std::string t = s;
13     ba::trim(t);
14     std::cout << "[" << t << "]\n"; // [Merhaba, Dünya]
15
16     // Büyük/küçük harf
17     std::cout << ba::to_upper_copy(t) << "\n"; // MERHABA, DUNYA
18     std::cout << ba::to_lower_copy(t) << "\n"; // merhaba, dunya
19
20     // Bölme (split) -> vektöre
21     std::vector<std::string> parts;
22     std::string csv = "elma,armut,kiraz,uzum";
23     ba::split(parts, csv, ba::is_any_of(","));
24     for (const auto& p : parts) std::cout << "- " << p << "\n";
25
26     // Birleştirme (join)
27     std::cout << ba::join(parts, " | ") << "\n";
28
29     // Değiştirme
30     std::string text = "kedi kedi kedi";
31     ba::replace_all(text, "kedi", "kopek");
32     std::cout << text << "\n"; // kopek kopek kopek
33
34     // Başlangıç/bitis/icerik kontrolü (büyük-küçük duyarsız)
35     std::cout << std::boolalpha

```

```

36         << ba::istarts_with("Merhaba", "MER") << " "
37         << ba::contains("Merhaba", "rha") << "\n";
38     return 0;
39 }

```

7.4 Boost.Tokenizer: Esnek Belirteçleme

`boost::tokenizer`, bir dizeyi belirteçlere (token) ayırmanın esnek bir yolunu sunar; farklı ayırıcı stratejileri (karakter, escape'li, CSV) takılabilir.

```

1  #include <boost/tokenizer.hpp>
2  #include <iostream>
3  #include <string>
4
5  int main() {
6      std::string s = "bir,iki;uc dort";
7
8      // Birden fazla ayırıcı karakter
9      boost::char_separator<char> separator(",; ");
10     boost::tokenizer<boost::char_separator<char>> tok(s, separator);
11
12     for (const auto& t : tok)
13         std::cout << "[" << t << "]";
14     std::cout << "\n";           // [bir][iki][uc][dort]
15
16     // CSV ayırıcı: tırnak ve escape'leri doğru işler
17     std::string csv = "\"Ali, Veli\",42,\"istanbul\"";
18     boost::tokenizer<boost::escaped_list_separator<char>> csvtok(csv);
19     for (const auto& field : csvtok)
20         std::cout << "Alan: " << field << "\n";
21     return 0;
22 }

```


KISIM 3

Konteynerler ve Veri Yapıları

8. Boost.Container: Gelişmiş Konteynerler

Boost.Container, standart konteynerlerin geliştirilmiş sürümlerini ve standartta bulunmayan yeni konteynerler sunar. Öne çıkanlar:

Konteyner	Özelliği
flat_map / flat_set	Sıralı vektör tabanlı; daha iyi önbellek yerelliği, daha az bellek. Arama $\mathcal{O}(\log n)$, ekleme $\mathcal{O}(n)$.
small_vector	Küçük boyutlar için yığın (stack) üzerinde depolama; tahsisi geciktirir.
static_vector	Sabit kapasiteli, hiç yığın tahsisi yapmayan vektör.
stable_vector	Elemanlara işaretçi/iteratör kararlılığı garanti eder.
devector	Çift uçtan büyüyeabilen vektör (deque + vector melezi).

```

1 #include <boost/container/flat_map.hpp>
2 #include <boost/container/small_vector.hpp>
3 #include <boost/container/static_vector.hpp>
4 #include <iostream>
5
6 int main() {
7     namespace bc = boost::container;
8
9     // flat_map: sıralı vektör tabanlı map. Az eleman + çok arama için ideal.
10    bc::flat_map<int, std::string> fm;
11    fm[3] = "uc"; fm[1] = "bir"; fm[2] = "iki";
12    for (const auto& [k, v] : fm) // otomatik sıralı: 1,2,3
13        std::cout << k << "=" << v << " ";
14    std::cout << "\n";
15
16    // small_vector<T, N>: ilk N eleman stack'te, tasma olursa heap'e gecer
17    bc::small_vector<int, 4> sv;
18    sv.push_back(1); sv.push_back(2); // heap tahsisi YOK (4'e kadar)
19    std::cout << "small_vector boyut: " << sv.size() << "\n";

```

```

20
21 // static_vector<T, N>: kapasite sabit N, ASLA heap kullanmaz
22 bc::static_vector<int, 3> stv;
23 stv.push_back(10); stv.push_back(20); stv.push_back(30);
24 // stv.push_back(40); // -> kapasite asıldı, tanımı/istisna
25 std::cout << "static_vector kapasite: " << stv.capacity() << "\n";
26 return 0;
27 }

```

Bilgi

flat_map, std::map'e göre çok daha iyi önbellek davranışı sergiler çünkü elemanlar bellekte bitişiktir. Okuma ağırlıklı, nadiren değişen sözlükler için idealdir. Ancak sık ekleme/silme yaparsanız her işlem $O(n)$ eleman kaydırıldığından std::map daha iyidir. C++23'te std::flat_map olarak standarda girmiştir.

9. Boost.MultiIndex: Çok İndeksli Konteynerler

Boost.MultiIndex, tek bir eleman kümesine *aynı anda birden fazla farklı indeksle* erişmenizi sağlar; tıpkı bir veritabanı tablosunun birden fazla indeksi olması gibi. Bir çalışan listesini hem ID'ye, hem isme, hem de maaşa göre sıralı tutmak isterseniz, veriyi kopyalamadan tek yapıda yönetirsiniz.

```

1 #include <boost/multi_index_container.hpp>
2 #include <boost/multi_index/ordered_index.hpp>
3 #include <boost/multi_index/member.hpp>
4 #include <iostream>
5 #include <string>
6
7 struct Employee {
8     int id;
9     std::string name;
10    double salary;
11    Employee(int i, std::string n, double m)
12        : id(i), name(std::move(n)), salary(m) {}
13 };
14
15 // İndeksleri isimlendirmek için etiketler
16 struct byId {};
17 struct byName {};
18 struct bySalary {};
19
20 namespace bmi = boost::multi_index;
21
22 // Uc farklı indeksli konteyner tanımla
23 using EmployeeTable = bmi::multi_index_container<
24     Employee,
25     bmi::indexed_by<
26         // 1) ID'ye göre eşsiz (unique) sıralı indeks
27         bmi::ordered_unique<
28             bmi::tag<byId>,
29             bmi::member<Employee, int, &Employee::id>>,
30         // 2) Isme göre (tekrar edebilir) sıralı indeks
31         bmi::ordered_non_unique<
32             bmi::tag<byName>,
33             bmi::member<Employee, std::string, &Employee::name>>,

```

```

34      // 3) Maasa gore sirali indeks
35      bmi::ordered_non_unique<
36          bmi::tag<bySalary>,
37          bmi::member<Employee, double, &Employee::salary>>
38      >
39  >;
40
41  int main() {
42      EmployeeTable table;
43      table.insert(Employee(3, "Ayse", 8500));
44      table.insert(Employee(1, "Mehmet", 7200));
45      table.insert(Employee(2, "Zeynep", 9100));
46
47      // ID'ye gore sirali gezinme
48      std::cout << "== ID sirasi ==\n";
49      for (const auto& c : table.get<byId>())
50          std::cout << c.id << " " << c.name << " " << c.salary << "\n";
51
52      // Isme gore arama
53      auto& nameIdx = table.get<byName>();
54      auto it = nameIdx.find("Zeynep");
55      if (it != nameIdx.end())
56          std::cout << "\nZeynep'in maasi: " << it->salary << "\n";
57
58      // Maasa gore sirali (en dusukten en yuksege)
59      std::cout << "\n== Maas sirasi ==\n";
60      for (const auto& c : table.get<bySalary>())
61          std::cout << c.salary << " -> " << c.name << "\n";
62      return 0;
63  }

```

İpucu

MultiIndex, bellekteki verileri bir mini-veritabanı gibi yönetmek istediğinizde muhteşemdir: LRU önbellekleri, oyun varlık yöneticileri, ağ bağlantı tabloları. `ordered`, `hashed`, `sequenced` ve `random_access` indeks türlerini karıştırabilirsiniz. Tek bir elemanı güncellemek tüm indeksleri otomatik tutarlı tutar.

10. Boost.Bimap: Çift Yönlü Haritalar

`boost::bimap`, hem anahtardan değere hem de değerden anahtara $\mathcal{O}(\log n)$ erişim sağlayan çift yönlü bir haritadır. Normal `std::map` yalnızca tek yönde arama yaparken, `bimap` her iki yönü de indeksler.

```

1  #include <boost/bimap.hpp>
2  #include <iostream>
3  #include <string>
4
5  int main() {
6      // Ulke <-> baskent iki yonlu esleme
7      using Bimap = boost::bimap<std::string, std::string>;
8      Bimap b;
9
10     b.insert({"Turkiye", "Ankara"});
11     b.insert({"Fransa", "Paris"});

```

```

12     b.insert({"Japonya", "Tokyo"});
13
14     // Sol -> sag (ulke -> baskent)
15     std::cout << "Turkiye'nin baskenti: "
16               << b.left.at("Turkiye") << "\n";
17
18     // Sag -> sol (baskent -> ulke)
19     std::cout << "Paris hangi ulkede: "
20               << b.right.at("Paris") << "\n";
21
22     // Tum eslemeleri gezme
23     for (const auto& c : b.left)
24         std::cout << c.first << " <-> " << c.second << "\n";
25     return 0;
26 }

```

Bilgi

Bimap, iki değer arasında birebir (one-to-one) ilişki olduğunda idealdir: kullanıcı adı ↔ ID, sembol ↔ kod, enum ↔ metin. İki ayrı std::map tutup senkron kalmaya çalışmaktan çok daha güvenlidir.

11. Boost.CircularBuffer: Dairesel Tampon

boost::circular_buffer, sabit kapasiteli, dolduğunda en eski elemanların üzerine yazan bir tampondur. Sabit boyutlu kayan pencereler, son N olayı tutma, ses/sinyal tamponları için idealdir.

```

1  #include <boost/circular_buffer.hpp>
2  #include <iostream>
3
4  int main() {
5      // Kapasite 3'lük daireysel tampon
6      boost::circular_buffer<int> cb(3);
7
8      cb.push_back(1);
9      cb.push_back(2);
10     cb.push_back(3);
11     std::cout << "Dolu mu? " << std::boolalpha << cb.full() << "\n"; // true
12
13     // Kapasite dolu; yeni eleman EN ESKİYİ (1) atar
14     cb.push_back(4); // [2, 3, 4]
15     cb.push_back(5); // [3, 4, 5]
16
17     std::cout << "İçerik: ";
18     for (int x : cb) std::cout << x << " "; // 3 4 5
19     std::cout << "\n";
20
21     std::cout << "En eski: " << cb.front() // 3
22               << ", en yeni: " << cb.back() << "\n"; // 5
23     return 0;
24 }

```

12. Boost.DynamicBitset: Dinamik Bit Kümeleri

`std::bitset` derleme zamanında sabit boyutludur; `boost::dynamic_bitset` ise çalışma zamanında boyutlandırılabilir. Bit maskeleri, kümeler, elek algoritmaları için idealdir.

```
1 #include <boost/dynamic_bitset.hpp>
2 #include <iostream>
3
4 int main() {
5     // Calisma zamanInda 16 bitlik kume
6     boost::dynamic_bitset<> bits(16);
7
8     bits.set(2);           // 2. biti 1 yap
9     bits.set(5);
10    bits.set(8);
11
12    std::cout << "Bitler: " << bits << "\n";           // ...0100100100
13    std::cout << "Kac bit 1: " << bits.count() << "\n"; // 3
14    std::cout << "Boyut: " << bits.size() << "\n";     // 16
15
16    // Bitset operatorler
17    boost::dynamic_bitset<> mask(16);
18    mask.set(2); mask.set(3);
19    auto intersection = bits & mask; // AND
20    std::cout << "Kesisim sayisi: " << intersection.count() << "\n"; // 1 (bit 2)
21
22    // Ilk 1 olan biti bul, sonrakileri gez
23    for (size_t i = bits.find_first();
24         i != boost::dynamic_bitset<>::npos;
25         i = bits.find_next(i)) {
26        std::cout << "Set bit @ " << i << "\n";
27    }
28    return 0;
29 }
```

KISIM 4

Eşzamanlılık ve Ağ Programlama

13. Boost.Thread: İş Parçacığı Yönetimi

Boost.Thread, `std::thread`'in atasıdır ve standartta *hâlâ bulunmayan* bazı özellikler sunar: `shared_mutex`'in daha zengin sürümleri, kesintiye uğratılabilir (interruptible) iş parçacıkları, `thread_group`, ve gelişmiş `future/promise` desteği (örn. `when_all`, `then` zincirlemesi).

```

1  #include <boost/thread.hpp>
2  #include <boost/thread/future.hpp>
3  #include <iostream>
4  #include <vector>
5
6  int compute(int x) {
7      boost::this_thread::sleep_for(boost::chrono::milliseconds(50));
8      return x * x;
9  }
10
11 int main() {
12     // 1) Temel iş parçacığı
13     boost::thread t([]{
14         std::cout << "İş parçacığından merhaba\n";
15     });
16     t.join();
17
18     // 2) thread_group: birden fazla iş parçacığını birlikte yönet
19     boost::thread_group group;
20     for (int i = 0; i < 4; ++i)
21         group.create_thread([i]{
22             std::cout << "İşci " << i << " çalışıyor\n";
23         });
24     group.join_all();    // hepsini bekle
25
26     // 3) future/promise ile asenkron sonuç
27     boost::future<int> f = boost::async(boost::launch::async, compute, 7);
28     std::cout << "7^2 = " << f.get() << "\n";    // 49
29
30     // 4) .then() ile sürekli zincirleme (Boost'a özgü, std'de yok)
31     boost::future<int> chain =
32         boost::async([]{ return 10; })
33         .then([](boost::future<int> prev){
34             return prev.get() * 2;    // 20
35         });

```

```

36     std::cout << "Zincir sonucu: " << chain.get() << "\n";
37     return 0;
38 }

```

Dikkat

Boost.Thread'in .then(), when_all, when_any sürekli zincirleme (continuation) özellikleri için BOOST_THREAD_PROVIDES_FUTURE_CONTINUATION gibi makrolar veya doğru derleme bayrakları gerekebilir. std::future bu özelliklere sahip değildir; bu yüzden asenkron zincirleme istiyorsanız Boost.Thread hâlâ geçerlidir.

13.1 Karşılıklı Dışlama ve Kilitler

```

1  #include <boost/thread.hpp>
2  #include <iostream>
3
4  class SafeCounter {
5      mutable boost::shared_mutex mtx_;    // okuyucu-yazıcı kilidi
6      int value_ = 0;
7  public:
8      void increment() {
9          // Yazma: özel (exclusive) kilit
10         boost::unique_lock<boost::shared_mutex> lk(mtx_);
11         ++value_;
12     }
13     int read() const {
14         // Okuma: paylaşımlı (shared) kilit -- çoklu okuyucu aynı anda
15         boost::shared_lock<boost::shared_mutex> lk(mtx_);
16         return value_;
17     }
18 };
19
20 int main() {
21     SafeCounter counter;
22     boost::thread_group group;
23     for (int i = 0; i < 10; ++i)
24         group.create_thread([&]{
25             for (int j = 0; j < 1000; ++j) counter.increment();
26         });
27     group.join_all();
28     std::cout << "Toplam: " << counter.read() << "\n";    // 10000
29     return 0;
30 }

```

14. Boost.Asio: Asenkron Giriş/Çıkış ve Ağ

Boost.Asio, C++'ta ağ ve düşük seviye asenkron G/Ç programlamanın *filii standardıdır*. TCP/UDP soketleri, zamanlayıcılar, seri portlar, ve platformdan bağımsız bir olay döngüsü (event loop) sunar. C++ Networking TS'nin temelidir. Asio'nun merkezinde `io_context` (olay döngüsü) bulunur; tüm asenkron işlemler ona kayıtlanır.

14.1 Temel Kavramlar: `io_context` ve Zamanlayıcılar

```

1  #include <boost/asio.hpp>

```

```

2 #include <iostream>
3
4 int main() {
5     boost::asio::io_context io;
6
7     // Asenkron zamanlayıcı: 2 saniye sonra tetiklenir
8     boost::asio::steady_timer timer(io, boost::asio::chrono::seconds(2));
9
10    timer.async_wait([](const boost::system::error_code& ec){
11        if (!ec) std::cout << "2 saniye geçti!\n";
12    });
13
14    std::cout << "Bekleniyor...\n";
15    io.run(); // olay dongusu: kayıtlı tüm işler bitene kadar çalışır
16    std::cout << "io_context bitti\n";
17    return 0;
18 }

```

Bilgi

`io_context.run()` çağrısı, kayıtlı asenkron iş kalmayana kadar bloke olur ve bu sırada tamamlanan işlerin geri çağırma (callback) fonksiyonlarını çalıştırır. Asio'nun gücü, *tek bir iş parçacığında* binlerce eşzamanlı bağlantıyı yönetebilmesidir (proaktör deseni). Ağır yük için `io.run()`'ı birden fazla iş parçacığında çağırarak ölçekleyebilirsiniz.

14.2 Asenkron TCP Yankı (Echo) Sunucusu

Aşağıda, gelen her bağlantıyı kabul edip aldığı veriyi geri yollayan tam bir asenkron TCP sunucusu bulunmaktadır. Bu, Asio'nun gerçek gücünü gösteren kanonik örnektir.

```

1 #include <boost/asio.hpp>
2 #include <iostream>
3 #include <memory>
4
5 using boost::asio::ip::tcp;
6
7 // Her baglanti icin bir oturum. shared_ptr ile omru yonetilir:
8 // asenkron isler tamamlanana kadar nesne yasar (enable_shared_from_this).
9 class Session : public std::enable_shared_from_this<Session> {
10 public:
11     explicit Session(tcp::socket socket) : socket_(std::move(socket)) {}
12
13     void start() { read(); }
14
15 private:
16     void read() {
17         auto self = shared_from_this(); // omru uzat
18         socket_.async_read_some(
19             boost::asio::buffer(buffer_),
20             [this, self](boost::system::error_code ec, std::size_t n) {
21                 if (!ec) {
22                     write(n); // okunan veriyi geri yaz
23                 }
24                 // ec varsa oturum sessizce sonlanir (self referansi dusunce silinir)
25             });
26     }
27
28     void write(std::size_t length) {

```



```

29     auto self = shared_from_this();
30     boost::asio::async_write(
31         socket_,
32         boost::asio::buffer(buffer_, length),
33         [this, self](boost::system::error_code ec, std::size_t) {
34             if (!ec) read();          // yazma bitti, tekrar oku (dongu)
35         });
36 }
37
38 tcp::socket socket_;
39 char buffer_[1024];
40 };
41
42 // Yeni baglantilari kabul eden sunucu
43 class Server {
44 public:
45     Server(boost::asio::io_context& io, unsigned short port)
46         : acceptor_(io, tcp::endpoint(tcp::v4(), port)) {
47         doAccept();
48     }
49
50 private:
51     void doAccept() {
52         acceptor_.async_accept(
53             [this](boost::system::error_code ec, tcp::socket socket) {
54                 if (!ec) {
55                     std::cout << "Yeni baglanti: "
56                               << socket.remote_endpoint() << "\n";
57                     std::make_shared<Session>(std::move(socket))->start();
58                 }
59                 doAccept(); // bir sonraki baglantiyi kabul etmeye devam et
60             });
61     }
62     tcp::acceptor acceptor_;
63 };
64
65 int main() {
66     try {
67         boost::asio::io_context io;
68         Server server(io, 12345);
69         std::cout << "Yanki sunucusu 12345 portunda dinliyor...\n";
70         io.run();
71     } catch (const std::exception& e) {
72         std::cerr << "Hata: " << e.what() << "\n";
73     }
74     return 0;
75 }

```

```

1 # Derleme (Asio, System kutuphanesine baglanir)
2 g++ -std=c++17 echo_server.cpp -o echo_server -lboost_system -lpthread
3 ./echo_server &
4 # Baska bir terminalde test:
5 echo "merhaba asio" | nc localhost 12345
6 # Cikti: merhaba asio (sunucu geri yansitir)

```

Dikkat

Asio ile asenkron programlamada `shared_from_this` deseni kritiktir: bir asenkron işlem devam ederken nesnenin (oturumun) yok edilmemesini garanti eder. Geri çağırma lambdasına `self` kopyalayarak referans sayacını yüksek tutarsınız; işlem bitince `self` kapsam dışına çıkar ve nesne uygun şekilde temizlenir. Bu deseni atlarsanız sarkan işaretçi (dangling pointer) ve çökme yaşarsınız.

14.3 Asenkron TCP İstemcisi

```

1  #include <boost/asio.hpp>
2  #include <iostream>
3  #include <string>
4
5  using boost::asio::ip::tcp;
6
7  int main() {
8      try {
9          boost::asio::io_context io;
10
11         // Sunucu adresini coz (resolve)
12         tcp::resolver resolver(io);
13         auto endpoints = resolver.resolve("localhost", "12345");
14
15         // Baglan (senkron -- basitlik icin)
16         tcp::socket socket(io);
17         boost::asio::connect(socket, endpoints);
18
19         // Veri gonder
20         std::string message = "Merhaba sunucu!";
21         boost::asio::write(socket, boost::asio::buffer(message));
22
23         // Yaniti oku
24         char response[128];
25         std::size_t n = socket.read_some(boost::asio::buffer(response));
26         std::cout << "Sunucudan: " << std::string(response, n) << "\n";
27     } catch (const std::exception& e) {
28         std::cerr << "Istemci hatasi: " << e.what() << "\n";
29     }
30     return 0;
31 }
```

14.4 Modern Yaklaşım: Coroutine'lerle Asio (C++20)

Asio, C++20 coroutine'leriyle birlikte kullanıldığında asenkron kodu *senkron kod kadar okunur* hale getirir. `co_await` ile geri çağırma cehennemi (callback hell) tamamen ortadan kalkar.

```

1  #include <boost/asio.hpp>
2  #include <boost/asio/co_spawn.hpp>
3  #include <boost/asio/detached.hpp>
4  #include <iostream>
5
6  using boost::asio::ip::tcp;
7  using boost::asio::awaitable;
8  using boost::asio::use_awaitable;
9  namespace this_coro = boost::asio::this_coro;
10
11  // Coroutine tabanlı yanki oturumu -- geri caGirma yok, duz akIs!
```

```

12 awaitable<void> echo(tcp::socket socket) {
13     char data[1024];
14     try {
15         for (;;) {
16             // co_await: asenkron ama senkron gorunumlu
17             std::size_t n = co_await socket.async_read_some(
18                 boost::asio::buffer(data), use_awaitable);
19             co_await boost::asio::async_write(
20                 socket, boost::asio::buffer(data, n), use_awaitable);
21         }
22     } catch (const std::exception& e) {
23         std::cout << "Oturum bitti: " << e.what() << "\n";
24     }
25 }
26
27 awaitable<void> listener() {
28     auto exec = co_await this_coro::executor;
29     tcp::acceptor acceptor(exec, {tcp::v4(), 12345});
30     for (;;) {
31         tcp::socket socket = co_await acceptor.async_accept(use_awaitable);
32         // Her baglanti icin ayri bir coroutine baslat
33         boost::asio::co_spawn(exec, echo(std::move(socket)),
34             boost::asio::detached);
35     }
36 }
37
38 int main() {
39     boost::asio::io_context io;
40     boost::asio::co_spawn(io, listener(), boost::asio::detached);
41     std::cout << "Coroutine yanki sunucusu 12345'te...\n";
42     io.run();
43     return 0;
44 }

```

```

1 # C++20 ve coroutine destegi gerekir
2 g++ -std=c++20 -fcoroutines coro_echo.cpp -o coro_echo \
3     -lboost_system -lpthread

```

İpucu

Coroutine tabanlı Asio, modern C++ ağ programlmasının geleceğidir. Yukarıdaki kod, önceki geri çağırılmalı yankı sunucusuyla aynı işi yapar ama akış çok daha nettir: `co_await` her satırda “burada asenkron bekle, sonra devam et” anlamına gelir. Hata yönetimi de normal `try/catch` ile yapılır. Yeni Asio projelerine coroutine’lerle başlamanızı öneririm.

15. Boost.Atomic ve Boost.Lockfree

`boost::lockfree`, kilit kullanmadan (lock-free) çalışan eşzamanlı veri yapıları sunar: `queue`, `stack`, `spsc_queue`. Yüksek performanslı, düşük gecikmeli sistemlerde (finans, oyun, ses) kilitlerin getirdiği bekleme ve öncelik tersine dönmesi (priority inversion) sorunlarından kaçınmak için kullanılır.

```

1 #include <boost/lockfree/queue.hpp>
2 #include <boost/thread.hpp>
3 #include <atomic>
4 #include <iostream>

```

```

5
6 int main() {
7     // Kilitli kuyruk: çoklu üretici/tüketici güvenli
8     boost::lockfree::queue<int> queue(128);
9     std::atomic<bool> done{false};
10    std::atomic<long> total{0};
11
12    // Üretici: 0..9999 arası değerleri kuyruğa koyar
13    boost::thread producer([&]{
14        for (int i = 0; i < 10000; ++i)
15            while (!queue.push(i)) {} // kuyruk doluyorsa tekrar dene
16        done = true;
17    });
18
19    // Tüketici: kuyruktan çekip toplar
20    boost::thread consumer([&]{
21        int value;
22        while (!done || !queue.empty()) {
23            while (queue.pop(value))
24                total += value;
25        }
26    });
27
28    producer.join();
29    consumer.join();
30    std::cout << "Toplam: " << total << "\n"; // 49995000
31    return 0;
32 }

```

```

1 g++ -std=c++17 lockfree.cpp -o lockfree -lboost_thread -lboost_system -lpthread

```

SPSC Kuyruğu

Tek üretici-tek tüketici (single producer, single consumer) senaryonuz varsa `boost::lockfree::spsc_queue` çok daha hızlıdır; genel `queue`'nın CAS (compare-and-swap) döngülerine gerek duymaz. Ses işleme ve gerçek zamanlı hatlarda tercih edilir.

KISIM 5

Metin Ayırıştırma ve Veri İşleme

16. Boost.Regex: Düzenli İfadeler

Boost.Regex, `std::regex`'in atasıdır. Bugün `std::regex` standartta olsa da, Boost.Regex genellikle *daha hızlıdır*, Perl uyumluluğu daha iyidir ve Unicode (ICU ile) desteği daha güçlüdür.

```

1  #include <boost/regex.hpp>
2  #include <iostream>
3  #include <string>
4
5  int main() {
6      std::string text = "iletisim: ali@example.com ve veli@test.org";
7
8      // E-posta yakalayan duzenli ifade
9      boost::regex email(R"((\w+)@(\w+\.\w+))");
10
11     // Tum eslesmeleri gez
12     boost::sregex_iterator it(text.begin(), text.end(), email);
13     boost::sregex_iterator end;
14     for (; it != end; ++it) {
15         boost::smatch e = *it;
16         std::cout << "Tam: " << e[0]           // ali@example.com
17                   << " | kullanicisi: " << e[1] // ali
18                   << " | alan: " << e[2] << "\n"; // example.com
19     }
20
21     // Degistirme (replace) -- geri referans ile
22     std::string masked = boost::regex_replace(
23         text, email, "[gizli]@$2");
24     std::cout << masked << "\n";
25     return 0;
26 }

```

```
1 g++ -std=c++17 regex_example.cpp -o regex_example -lboost_regex
```

17. Boost.PropertyTree: JSON, XML, INI Ayırıştırma

`boost::property_tree`, hiyerarşik yapılandırma verilerini (JSON, XML, INI, INFO) tek bir ağaç yapısında tutar ve bu formatlar arasında dönüşüm yapar. Basit yapılandırma dosyaları için hızlı bir çözümdür.

```

1  #include <boost/property_tree/ptree.hpp>
2  #include <boost/property_tree/json_parser.hpp>
3  #include <iostream>
4  #include <sstream>
5
6  namespace pt = boost::property_tree;
7
8  int main() {
9      // JSON'dan bir property tree olustur
10     std::string json = R"({
11         "sunucu": {
12             "host": "localhost",
13             "port": 8080,
14             "aktif": true
15         },
16         "kullanici": ["ali", "veli", "ayse"]
17     })";
18
19     pt::ptree tree;
20     std::istringstream input(json);
21     pt::read_json(input, tree);
22
23     // Deger okuma (nokta ile yol belirtme)
24     std::string host = tree.get<std::string>("sunucu.host");
25     int port = tree.get<int>("sunucu.port");
26     bool active = tree.get<bool>("sunucu.aktif");
27     std::cout << host << ":" << port
28         << " (aktif=" << std::boolalpha << active << ")\n";
29
30     // Varsayilanli okuma (anahtar yoksa)
31     int timeout = tree.get<int>("sunucu.timeout", 30);
32     std::cout << "Zaman asimi: " << timeout << "\n"; // 30 (yok, varsayilan)
33
34     // Dizi (array) gezme
35     std::cout << "Kullanici: ";
36     for (const auto& [key, value] : tree.get_child("kullanici"))
37         std::cout << value.get_value<std::string>() << " ";
38     std::cout << "\n";
39
40     // Degistir ve tekrar JSON yaz
41     tree.put("sunucu.port", 9090);
42     pt::write_json(std::cout, tree);
43     return 0;
44 }
```

Dikkat

`property_tree` her değeri dahili olarak *metin* olarak saklar; bu yüzden JSON'un sayı, boolean ve dizi türlerini tam olarak korumaz (örneğin diziler garip biçimde temsil edilir). Ciddi JSON işleri için bir sonraki bölümdeki `boost::json`'u kullanın. `PropertyTree`'yi basit INI/config dosyaları ve formatlar arası hızlı dönüşüm için saklayın.

18. Boost.JSON: Modern JSON İşleme

Boost.JSON (Boost 1.75+), yüksek performanslı, standartlara uygun, *gerçek* bir JSON kütüphanesidir. PropertyTree'nin aksine JSON tiplerini tam korur (sayı, string, bool, null, dizi, nesne) ve çok hızlıdır.

```

1  #include <boost/json.hpp>
2  #include <iostream>
3  #include <string>
4
5  namespace json = boost::json;
6
7  int main() {
8      // 1) Metinden ayrıştırma (parse)
9      std::string text = R"({
10         "isim": "Ada",
11         "yas": 36,
12         "diller": ["C++", "Python"],
13         "aktif": true
14     })";
15
16     json::value v = json::parse(text);
17     json::object& obj = v.as_object();
18
19     std::cout << "İsim: " << obj["isim"].as_string() << "\n";
20     std::cout << "Yas: " << obj["yas"].as_int64() << "\n";
21     std::cout << "Aktif: " << std::boolalpha << obj["aktif"].as_bool() << "\n";
22
23     // Dizi gezme
24     std::cout << "Diller: ";
25     for (const auto& d : obj["diller"].as_array())
26         std::cout << d.as_string() << " ";
27     std::cout << "\n";
28
29     // 2) Programatik olarak JSON inşa etme
30     json::object output;
31     output["urun"] = "Klavye";
32     output["fiyat"] = 450.0;
33     output["stok"] = true;
34     json::array tags = {"mekanik", "rgb"};
35     output["etiketler"] = tags;
36
37     // 3) Metne serileştirme (serialize)
38     std::cout << "Uretilen JSON:\n" << json::serialize(output) << "\n";
39     return 0;
40 }

```

```
1 g++ -std=c++17 json_example.cpp -o json_example -lboost_json
```

İpucu

Boost.JSON, `value_to<T>` ve `value_from<T>` ile kendi C++ tiplerinizi doğrudan JSON'a/JSON'dan dönüştürebilir (kullanıcı tanımlı dönüşümlerle). Ayrıca akış (streaming) ayrıştırıcısı sayesinde belleğe sığmayan devasa JSON dosyalarını parça parça işleyebilir. Yeni projelerde JSON için PropertyTree yerine Boost.JSON'u tercih edin.

19. Boost.Spirit: Ayrıştırıcı Üretici

Boost.Spirit, doğrudan C++ içinde, EBNF'ye benzer bir sözdizimiyle ayrıştırıcı (parser) yazmanızı sağlayan güçlü bir kütüphanedir. Dilbilgisi kurallarını C++ ifadeleri olarak yazarsınız; Spirit bunları derleme zamanında bir ayrıştırıcıya dönüştürür. Küçük diller (DSL), yapılandırma formatları ve protokol ayrıştırma için idealdir.

```

1 #include <boost/spirit/home/x3.hpp>
2 #include <iostream>
3 #include <string>
4 #include <vector>
5
6 namespace x3 = boost::spirit::x3;
7
8 int main() {
9     // Virgülle ayrılmış sayıları ayrıştır: "1.5, 2.0, 3.14"
10    std::string input = "1.5, 2.0, 3.14, 42";
11    std::vector<double> numbers;
12
13    auto first = input.begin();
14    auto end = input.end();
15
16    // Dilbilgisi: double (% ile ayrılmış) -- boşlukları atla
17    bool success = x3::phrase_parse(
18        first, end,
19        x3::double_ % ',', // ', ' ile ayrılmış double listesi
20        x3::space,         // atlanacak boşluk kuralı
21        numbers,           // sonuç buraya toplanır (attribute)
22    );
23
24    if (success && first == end) {
25        std::cout << "Ayrıştırıldı (" << numbers.size() << " sayı): ";
26        double sum = 0;
27        for (double s : numbers) { std::cout << s << " "; sum += s; }
28        std::cout << "\nToplam: " << sum << "\n";
29    } else {
30        std::cout << "Ayrıştırma başarısız\n";
31    }
32    return 0;
33 }
```

Bilgi

Spirit X3 (yukarıdaki modern sürüm) header-only'dir ve derleme zamanında çalışır. `x3::double_ % ', '` ifadesi, “virgülle ayrılmış bir veya daha fazla double” anlamına gelir. Spirit çok güçlüdür ama ağır şablon kullanımı nedeniyle derleme süresi ve hata mesajları zorlayıcı olabilir. Karmaşık dilbilgileri için harika, ama basit ayrıştırmalar için `std::from_chars` veya Tokenizer yeterli olabilir.

KISIM 6

Sistem, Matematik ve Graflar

20. Boost.Filesystem: Dosya Sistemi İşlemleri

Boost.Filesystem, `std::filesystem`'in (C++17) atasıdır. Dosya ve dizin işlemlerini platformdan bağımsız biçimde yapar: yol manipülasyonu, gezinme, oluşturma/silme, boyut/zaman sorgulama.

```

1  #include <boost/filesystem.hpp>
2  #include <iostream>
3
4  namespace fs = boost::filesystem;
5
6  int main() {
7      fs::path p = "/tmp/boost_test";
8
9      // Dizin olustur
10     fs::create_directories(p / "alt/dizin");
11
12     // Yol manipulasyonu
13     fs::path file = p / "veri.txt";
14     std::cout << "Ust dizin: " << file.parent_path() << "\n";
15     std::cout << "Dosya adi: " << file.filename() << "\n";
16     std::cout << "Uzanti: " << file.extension() << "\n";
17
18     // Varlik ve tur kontrolu
19     std::cout << "Var mi? " << std::boolalpha << fs::exists(p) << "\n";
20     std::cout << "Dizin mi? " << fs::is_directory(p) << "\n";
21
22     // Dizini ozyinelemeli (recursive) gez
23     std::cout << "\nIcerik:\n";
24     for (const auto& entry : fs::recursive_directory_iterator(p))
25         std::cout << " " << entry.path()
26             << (fs::is_directory(entry) ? " [dizin]" : " [dosya]") << "\n";
27
28     // Temizlik
29     std::uintmax_t removed = fs::remove_all(p);
30     std::cout << "\n" << removed << " oge silindi\n";
31     return 0;
32 }
```

```
g++ -std=c++17 fs_example.cpp -o fs_example -lboost_filesystem -lboost_system
```

İpucu

C++17 hedefliyorsanız, çoğu durumda `std::filesystem` yeterlidir ve bağımlılık gerektirmez. Boost.Filesystem'i, C++14/eski derleyici desteklemeniz gerekiyorsa veya `std`'de henüz olmayan bazı ek yardımcılarına (örn. `unique_path`, ek durum sorguları) ihtiyaç duyuyorsanız tercih edin. API'ler neredeyse birebir aynı olduğundan geçiş kolaydır.

21. Boost.DateTime ve Boost.Chrono

Boost.DateTime, tarih ve saat işlemleri için zengin bir kütüphanedir: takvim tarihleri, zaman aralıkları (`duration`), zaman dilimleri ve biçimlendirme. `std::chrono`'nun (C++11/20) sunmadığı takvim ve zaman dilimi özellikleri için hâlâ değerlidir.

```

1 #include <boost/date_time/gregorian/gregorian.hpp>
2 #include <boost/date_time/posix_time/posix_time.hpp>
3 #include <iostream>
4
5 int main() {
6     namespace greg = boost::gregorian;
7     namespace pt    = boost::posix_time;
8
9     // Tarih oluşturma
10    greg::date today(2026, greg::Jul, 19);
11    greg::date newYear(2026, greg::Jan, 1);
12
13    std::cout << "Bugün: " << today << "\n";
14    std::cout << "Haftanın günü: " << today.day_of_week() << "\n";
15    std::cout << "Yilin kacinci günü: " << today.day_of_year() << "\n";
16
17    // Tarih aritmetigi
18    greg::date_duration diff = today - newYear;
19    std::cout << "Yilbasından bu yana: " << diff.days() << " gün\n";
20
21    greg::date future = today + greg::days(100);
22    std::cout << "100 gün sonra: " << future << "\n";
23
24    // Zaman (posix_time)
25    pt::ptime now(today, pt::hours(14) + pt::minutes(30));
26    std::cout << "Zaman damgasi: " << now << "\n";
27
28    pt::time_duration duration = pt::hours(2) + pt::minutes(45) + pt::seconds(30);
29    std::cout << "Sure: " << duration
30              << " = " << duration.total_seconds() << " saniye\n";
31    return 0;
32 }

```

```
g++ -std=c++17 datetime_example.cpp -o datetime_example -lboost_date_time
```

22. Boost.ProgramOptions: Komut Satırı Ayrıştırma

Boost.ProgramOptions, komut satırı argümanlarını, ortam değişkenlerini ve yapılandırma dosyalarını profesyonelce ayrıştırır. Otomatik yardım metni, tip dönüşümü, varsayılan değerler ve doğrulama

sağlar.

```

1 #include <boost/program_options.hpp>
2 #include <iostream>
3 #include <string>
4
5 namespace po = boost::program_options;
6
7 int main(int argc, char* argv[]) {
8     // Secenekleri tanImla
9     po::options_description desc("Kullanım");
10    desc.add_options()
11        ("help,h", "Bu yardım metnini goster")
12        ("isim,n", po::value<std::string>()->default_value("Dunya"),
13            "Selamlanacak isim")
14        ("tekrar,t", po::value<int>()->default_value(1),
15            "Kac kez tekrarlansin")
16        ("sesli,v", po::bool_switch(), "Ayrintili cikti");
17
18    // Ayristir
19    po::variables_map vm;
20    try {
21        po::store(po::parse_command_line(argc, argv, desc), vm);
22        po::notify(vm);
23    } catch (const po::error& e) {
24        std::cerr << "Hata: " << e.what() << "\n";
25        return 1;
26    }
27
28    // --help verildiyse yardimi goster ve cik
29    if (vm.count("help")) {
30        std::cout << desc << "\n";
31        return 0;
32    }
33
34    // Degerleri kullan
35    std::string name = vm["isim"].as<std::string>();
36    int repeat = vm["tekrar"].as<int>();
37    bool verbose = vm["sesli"].as<bool>();
38
39    if (verbose) std::cout << "[ayrintili mod acik]\n";
40    for (int i = 0; i < repeat; ++i)
41        std::cout << "Merhaba, " << name << "!\n";
42    return 0;
43 }

```

```

1 g++ -std=c++17 opts.cpp -o opts -lboost_program_options
2 ./opts --isim Ada --tekrar 3 -v
3 # [ayrintili mod acik]
4 # Merhaba, Ada!
5 # Merhaba, Ada!
6 # Merhaba, Ada!
7 ./opts --help          # Otomatik uretilen yardım metnini gosterir

```

İpucu

ProgramOptions ile aynı seçenekleri hem komut satırından hem de bir .cfg dosyasından okuyabilirsiniz (parse_config_file). Komut satırı, config dosyasını geçersiz kılacak şekilde katmanlayabilirsiniz; bu, gerçek uygulamalarda standart bir yapılandırma stratejisidir.

23. Boost.Interprocess: Süreçler Arası İletişim

Boost.Interprocess, süreçler arası iletişim (IPC) için paylaşımlı bellek, bellek eşlemeli dosyalar, mesaj kuyukları ve süreçler arası senkronizasyon ilkeleri sunar. Aynı makinedeki farklı süreçlerin veri paylaşması gereken yüksek performanslı senaryolarda kullanılır.

```

1 #include <boost/interprocess/managed_shared_memory.hpp>
2 #include <iostream>
3
4 namespace ip = boost::interprocess;
5
6 int main() {
7     // Önceki çalışmalardan kalmis olabilecek segmenti temizle
8     ip::shared_memory_object::remove("BoostShm");
9
10    // 64 KB'lik yönetilen paylasimli bellek segmenti olustur
11    ip::managed_shared_memory segment(
12        ip::create_only, "BoostShm", 65536);
13
14    // Paylasimli bellekte bir int dizisi olustur ("Sayilar" adiyla)
15    int* array = segment.construct<int>("Sayilar")[10](0);
16    for (int i = 0; i < 10; ++i) array[i] = i * i;
17
18    std::cout << "Paylasimli bellege yazildi.\n";
19
20    // Baska bir surec ayni segmenti su sekilde acardi:
21    // managed_shared_memory seg(open_only, "BoostShm");
22    // auto res = seg.find<int>("Sayilar");
23    // int* dizi = res.first; size_t n = res.second;
24
25    // Ayni surec icinde bulup okuyalim (dogrulama)
26    auto result = segment.find<int>("Sayilar");
27    if (result.first) {
28        std::cout << "Bulundu, " << result.second << " eleman: ";
29        for (std::size_t i = 0; i < result.second; ++i)
30            std::cout << result.first[i] << " ";
31        std::cout << "\n";
32    }
33
34    ip::shared_memory_object::remove("BoostShm"); // temizle
35    return 0;
36 }

```

```

1 # Linux'ta paylasimli bellek icin rt kutuphanesi gerekebilir
2 g++ -std=c++17 ipc_example.cpp -o ipc_example -lrt -lpthread

```

Dikkat

Paylaşımlı bellekte *ham işaretçi* saklamayın; farklı süreçlerde adresler farklı olacağından işaretçiler geçersiz olur. Bunun yerine Boost.Interprocess'in `offset_ptr` tipini ve paylaşımlı bellek uyumlu ayırıcılarını (`allocator`) kullanın. Segmenti kullandıktan sonra `remove` ile temizlemeyi unutmayın; aksi halde çekirdek nesnesi sistemde kalır.

24. Boost.Multiprecision: Yüksek Hassasiyetli Aritmetik

Boost.Multiprecision, `int` veya `double`'ın sınırlarını aşan sayılarla çalışmanızı sağlar: keyfi hassasiyetli tam sayılar, ondalıklar ve rasyonel sayılar. Kriptografi, bilimsel hesaplama ve tam sayı taşmasının kabul edilemez olduğu durumlar için kritiktir.

```

1  #include <boost/multiprecision/cpp_int.hpp>
2  #include <boost/multiprecision/cpp_dec_float.hpp>
3  #include <iostream>
4
5  namespace mp = boost::multiprecision;
6
7  // 100! -- normal int'te imkansız (tasar), burada kesin
8  mp::cpp_int factorial(int n) {
9      mp::cpp_int result = 1;
10     for (int i = 2; i <= n; ++i)
11         result *= i;
12     return result;
13 }
14
15 int main() {
16     // Keyfi hassasiyetli tam sayı
17     std::cout << "100! = " << factorial(100) << "\n";
18
19     // Çok büyük uslu sayı
20     mp::cpp_int big = 1;
21     for (int i = 0; i < 200; ++i) big *= 2;    // 2^200
22     std::cout << "2^200 = " << big << "\n";
23
24     // 50 basamak hassasiyetli ondalık
25     using dec50 = mp::number<mp::cpp_dec_float<50>>;
26     dec50 pi = dec50("3.14159265358979323846264338327950288419716939937510");
27     dec50 radius = 100;
28     std::cout << "Alan = " << pi * radius * radius << "\n";
29
30     // Çok hassasiyetli karekok
31     dec50 two = 2;
32     std::cout << "sqrt(2) = " << sqrt(two) << "\n";
33     return 0;
34 }

```

Bilgi

`cpp_int` header-only'dir ve harici bağımlılık gerektirmez. Daha da yüksek performans için Boost, GMP (`gmp_int`) ve MPFR (`mpfr_float`) arka uçlarını da destekler; bunlar ilgili sistem kütüphanelerine bağlanmayı gerektirir ama devasa sayılarda çok daha hızlıdır. Rasyonel sayılar için `cpp_rational` kesin kesir aritmetiği sunar.

25. Boost.Graph (BGL): Graf Algoritmaları

Boost Graph Library (BGL), C++'ın en kapsamlı graf kütüphanesidir. Graf gösterimlerini (komşuluk listesi/matrisi) ve üzerlerinde çalışan onlarca klasik algoritmayı (BFS, DFS, Dijkstra, Bellman-Ford, Kruskal, Prim, topolojik sıralama, güçlü bağlı bileşenler) sağlar. Genel programlama (generic programming) tasarımı sayesinde kendi graf tipleriniz üzerinde bile çalışabilir.

25.1 Graf Oluşturma ve Gezinme

```

1 #include <boost/graph/adjacency_list.hpp>
2 #include <boost/graph/breadth_first_search.hpp>
3 #include <iostream>
4 #include <vector>
5
6 using namespace boost;
7
8 int main() {
9     // Yonsuz graf: komşuluk listesi tabanlı
10    using Graph = adjacency_list<vecS, vecS, undirectedS>;
11    Graph g(6); // 6 dugum (0..5)
12
13    // Kenar ekle
14    add_edge(0, 1, g);
15    add_edge(0, 2, g);
16    add_edge(1, 3, g);
17    add_edge(2, 3, g);
18    add_edge(3, 4, g);
19    add_edge(4, 5, g);
20
21    std::cout << "Dugum sayisi: " << num_vertices(g) << "\n";
22    std::cout << "Kenar sayisi: " << num_edges(g) << "\n";
23
24    // Tum kenarlari yazdir
25    std::cout << "Kenarlar: ";
26    auto [e_first, e_end] = edges(g);
27    for (auto it = e_first; it != e_end; ++it)
28        std::cout << source(*it, g) << "-" << target(*it, g) << " ";
29    std::cout << "\n";
30
31    // Bir dugumun komşularini gez
32    std::cout << "0'in komşulari: ";
33    auto [k_first, k_end] = adjacent_vertices(0, g);
34    for (auto it = k_first; it != k_end; ++it)
35        std::cout << *it << " ";
36    std::cout << "\n";
37    return 0;
38 }
```

25.2 Dijkstra ile En Kısa Yol

BGL'nin gerçek gücü hazır algoritmalarındadır. Aşağıda ağırlıklı bir grafta en kısa yolları Dijkstra ile buluyoruz:

```

1 #include <boost/graph/adjacency_list.hpp>
2 #include <boost/graph/dijkstra_shortest_paths.hpp>
3 #include <iostream>
4 #include <vector>
```

```

5
6 using namespace boost;
7
8 int main() {
9     // Kenar ağırlığı taşıyan yönlü graf
10    using Graph = adjacency_list<
11        vecS, vecS, directedS,
12        no_property,
13        property<edge_weight_t, int>>;    // kenarlarda ağırlık
14
15    Graph g(5);
16    // (kaynak, hedef, ağırlık)
17    add_edge(0, 1, 10, g);
18    add_edge(0, 2, 5, g);
19    add_edge(2, 1, 3, g);
20    add_edge(1, 3, 1, g);
21    add_edge(2, 3, 9, g);
22    add_edge(3, 4, 4, g);
23
24    // Sonuç dizileri: her düğüme uzaklık ve önceki düğüm
25    std::vector<int> distance(num_vertices(g));
26    std::vector<Graph::vertex_descriptor> previous(num_vertices(g));
27
28    auto start = vertex(0, g);
29
30    // Dijkstra'yi çalıştır
31    dijkstra_shortest_paths(g, start,
32        predecessor_map(&previous[0]).distance_map(&distance[0]));
33
34    // Sonuçları yazdır
35    std::cout << "0'dan en kısa uzaklıklar:\n";
36    for (std::size_t i = 0; i < distance.size(); ++i)
37        std::cout << "   Düğüm " << i << ": " << distance[i] << "\n";
38
39    // 0 -> 4 yolunu geri izle
40    std::cout << "0'dan 4'e yol: ";
41    std::vector<int> path;
42    for (int v = 4; v != start; v = previous[v]) path.push_back(v);
43    path.push_back(0);
44    for (auto it = path.rbegin(); it != path.rend(); ++it)
45        std::cout << *it << (it + 1 != path.rend() ? " -> " : "\n");
46    return 0;
47 }

```

```

1 # BGL cöğunlukla header-only'dir; bu örnekler bağlama gerektirmez
2 g++ -std=c++17 dijkstra.cpp -o dijkstra
3 ./dijkstra
4 # 0'dan en kısa uzaklıklar:
5 #   Düğüm 0: 0
6 #   Düğüm 1: 8   (0->2->1: 5+3)
7 #   Düğüm 2: 5
8 #   Düğüm 3: 9   (0->2->1->3: 5+3+1)
9 #   Düğüm 4: 13
10 # 0'dan 4'e yol: 0 -> 2 -> 1 -> 3 -> 4

```

Bilgi

BGL, C++ genel programlamanın bir yapıdır: algoritmalar graf gösteriminden tamamen bağımsızdır ve “property map” soyutlaması sayesinde düğüm/kenar verilerine esnek erişim sağlar. Öğrenme eğrisi diktir, ancak bir kez kavradığınızda son derece güçlüdür. Karmaşık grafik hesaplamaları için tekerleği yeniden icat etmek yerine BGL’yi değerlendirin.

26. Kapanış: Boost Ekosistemine Bakış

Bu rehber Boost’un en çok kullanılan kütüphanelerini kapsadı, ancak Boost bir buzdağdır. Karşılaşabileceğiniz diğer önemli kütüphaneler:

Kütüphane	Amaç
Boost.Hana	Modern (C++14+) metaprogramlama; MPL’nin halefi.
Boost.Mp11	Hafif, pratik tip-listesi metaprogramlama.
Boost.Geometry	Uzamsal algoritmalar, R-tree, coğrafi hesap.
Boost.Beast	Asio üzerine HTTP ve WebSocket.
Boost.Log	Esnek, ölçeklenebilir günlükleme (logging).
Boost.Serialization	Nesneleri diske/ağa serileştirme.
Boost.Fiber	Kullanıcı-alanı iş parçacıkları (M:N zamanlama).
Boost.Signals2	İş parçacığı güvenli sinyal/yuva (observer).
Boost.Coroutine2	Yığın tabanlı (stackful) coroutine’ler.
Boost.Test	Kapsamlı birim test çerçevesi.
Boost.Math	Özel fonksiyonlar, istatistik, dağılımlar.
Boost.uBLAS	Temel doğrusal cebir (matris/vektör).

İpucu

Öğrenme stratejisi: Boost’un tamamını öğrenmeye çalışmayın; bu ne mümkün ne de gerekli. Bunun yerine, bir problemle karşılaştığınızda “Bunun için bir Boost kütüphanesi var mı?” diye sorun. Genellikle vardır ve iyi test edilmiştir. Her kütüphanenin resmi dokümantasyonu, çalışan örneklerle doludur ve başlangıç noktanız olmalıdır: boost.org/doc/libs.

Bilgi

Modern C++ ve Boost dengesi: C++ standardı her sürümde büyüdükçe (C++20 concepts, ranges, coroutines; C++23 flat_map, expected), Boost’un bazı kütüphaneleri gereksizleşiyor. Yine de Asio, Spirit, Graph, Multiprecision, Geometry, Beast ve Interprocess gibi kütüphaneler standartta karşılığı olmayan, vazgeçilmez araçlar olmaya devam ediyor. Doğru yaklaşım: standartta varsa std’yi, yoksa Boost’u kullanın.

Rehberin Sonu

Boost 1.85+ • C++17/20 • İyi kodlamalar!